P1967R9

`#embed` - a scannable, tooling-friendly binary resource inclusion mechanism

Published Proposal, 2022-10-15

Authors:

JeanHeyd Meneide

Shepherd (Shepherd's Oasis LLC)

Latest:

https://thephd.dev/_vendor/future_cxx/papers/d1967.html

Paper Source:

GitHub ThePhD/future_cxx

Implementation:

GitHub ThePhD/embed

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

EWG

Abstract

Pulling binary data into a program often involves external tools and build system coordination. Many programs need binary data such as images, encoded text, icons and other data in a specific format. Current state of the art for working with such static data in C includes creating files which contain solely string literals, directly invoking the linker to create data blobs to access through carefully named extern variables, or generating large brace-delimited lists of integers to place into arrays. As binary data has grown larger, these approaches have begun to have drawbacks and issues scaling. From parsing 5 megabytes worth of integer literal expressions into AST nodes to arbitrary string literal length limits in compilers, portably putting binary data in a C program has become an arduous task that taxes build infrastructure and compilation memory and time. This proposal provides a flexible preprocessor directive for making this data available to the user in a straightforward manner.

Table of Contents

§ 1. Changelog

§ 1.1. Revision 9 - October 15th, 2022

— The C edition of the `#embed` paper was accepted into C23.

— It was accepted with all of the optional components (`if_empty`, `__has_embed`).

— These will be kept optional in this paper, but this information may inform the C++ Standards Committee of its decisions.

§ 1.2. Revision 8 - July 15th, 2022

— Remove `__` spelling from C++ proposal (not the C proposal): C++ does not have a rule about `__nodiscard__` being considered identical to `nodiscard` in spelling, as explained in § 4.2.1.4 Why is __param_name__ part of the grammar?.

— Document discussion and reaction to discussion from June 2022 meeting in § 2.1 June 2022 Virtual C++ meeting.

— Add an example for `if_empty` with `limit(0)`.

— Add some explanations and showings for `__has_embed` versus `suffix`/`prefix`/`if_empty`-style of parameters in § 6.1 __has_embed(…) == 2, suffix/prefix/if_empty parameters, both, or neither?.

§ 1.3. Revision 7 - June 23rd, 2022

— Add letter of support sent to the C Committee.

— Add brief history.

— Wording improvements:

  — Italicize definitions only once, not repeatedly, unless they are Words of Power.

  — Remove redundant/poorly copied text for embed parameters.

  — Move "Notes" into "Recommended Practice" sections.

  — Use textual descriptions for some of the mathematics.

— Rename `is_empty` to `if_empty`, per C++ Standards Committee request.

— Make `prefix`/`suffix`/`if_empty` optional, so as to be polled for inclusion in the next revision.

— Remove all mentions of "attribute", and produce solely parameter types.

— Document discussion and reaction to discussion from June 2022 meeting in § 2.1 June 2022 Virtual C++ meeting.

§ 1.4. Revision 6 - May 12th, 2022

— Minor typo and grammar fixes in the wording.

§ 1.5. Revision 5 - April 12th, 2022

— Additional syntax changes based on feedback from Joseph Myers, Hubert Tong, Jens Maurer, other implementers, and users.

— Minor wording tweaks and typo clean up.

— An implementation available in Godbolt (since last revision as well and noted below).

— The paper's source code has been refactored:

  — Separated WG21 paper from WG14 paper.

  — Core paper together (rationale, reasoning), included in both C and C++ papers since rationale is identical.

— Changed `__has_embed` to match feedback from last standards meeting, nominally that an empty resource returns `2` instead of `1` (but both decay to a truthy value during preprocessor conditional

inclusion expressions). Modified the wording and the prose in § 4.4 __has_embed to match.

— The wording for the limit parameter (§ 7.3.5 Add a new sub-clause §15.4.2 under Resource Inclusion for Embed parameters [cpp.res.param]) adjusted to perform macro expansion, at least once. Exact wording may need help.

§ 1.6. Revision 4 - June 15th, 2021

— Vastly improve C++ wording after June 3rd, 2021 discussion.

— Change syntax after comments from implementers of scanners / dependency trackers, and comments from implementers that were supported by users.

— Add support for "named parameter" implementation extensions.

§ 1.7. Revision 3 - April 15th, 2021

— Added post C meeting fixes to prepare for hopeful success next meeting.

— Added 2 more examples to C and C++ wording.

— Vastly improved wording and reduced ambiguities in syntax and semantics.

— Fixed various wording issues.

§ 1.8. Revision 2 - October 25th, 2020

— Added post C++ meeting notes and discussion.

— Removed type or bit specifications from the `#embed` directive.

— Moved "Type Flexibility" section and related notes to the Appendix as they are now unpursued.

§ 1.9. Revision 1 - April 10th, 2020

— Added post C meeting notes and discussion.

— Added discussion of potential endianness.

— Improved wording section at the end to be more detailed in handling preprocessor (which does not understand types).

§ 1.10. Revision 0 - January 5th, 2020

— Initial release.

§ 2. Polls & Votes

The votes for the C++ Committee are as follows:

— SF: Strongly in Favor

— F: In Favor

— N: Neutral

— A: Against

— SA: Strongly Against

§ 2.1. June 2022 Virtual C++ meeting

"EWG encourages P1967 to define the form of vendor extensions as parameters to `#embed`?"

SF	F	N	A	SA
4	4	3	1	0

This was the result of consensus. The extensive discussion also made it clear that we must make sure that unrecognized embed parameters, due to them changing how an initializer may be formed, must be considered ill-formed. Users may get around this by using `__has_embed`. To dispel the notion that they may be optional, frontmatter wording was added to § 7.3.2 Add to the control-line production in §15.1 Preamble [cpp.pre] a new grammar production, as well as a supporting embed-parameter-seq production to make it clear the expectations.

Part of the discussion during this meeting was also whether or not the case for emptiness was useful. We moved the empty-based parameters to **OPTIONAL** pieces of wording, and expect to forward each of these on independent votes asides from the base proposal. This captures the sentiment of folks who may not have spoken up a lot during the meeting but nevertheless felt uneasy: we can simply go with whatever the poll says next meeting.

We took the feedback to rename `is_empty` to `if_empty`, since it is a better name for a "do-something-if-predicate-is-true" style attribute.

§ 2.2. July 2021 Virtual C++ meeting

No votes were taken at this meeting, since it was mostly directional and about the changing of the syntax to better fit tools and scanners. In particular, it was more or less unanimously encouraged to:

— re-do the syntax to be `#embed header-name additional-tokens...` instead of `#embed limit-parameter header-name`;

— the `additional-tokens` should be reshaped into a parameter specification, giving both standard parameters (such as making a named `limit(integer-constant)` argument) and implementation-defined ones (such as `clang::element_type(short)`);

— and, the wording should include some recommendation or specification for "as-if by fread" to make wording easier.

All of these recommendations were incorporated below.

§ 2.3. September 2020 Virtual C++ EWG Meeting

"We want `#embed [optional limit] header-name` (no type name, no other specification) as a feature."

SF	F	N	A	SA
2	16	3	0	1

This vote gained the most consensus in the Committee. While there were some individuals who wanted to be able to specify a type, there was stronger interest in not specifying a type at all and always producing a list of integer literals suitable to be used anywhere an `comma-separated list` was valid.

"We want to explore allowing an optional sequence of tokens to specify a type to `#embed`."

SF	F	N	A	SA
1	9	4	4	3

Further need was also expressed for `constexpr` of different types of variables, so we would rather focus that ability into a sister feature, `std::embed`. There was also an expression to augment `std::bitcast<...>(...)` to handle arrays of data, which would be a follow-on proposal. There was a great amount of interest in the `std::bitcast` direction, which means a paper should be written to follow up on it.

§ 2.4. April 2020 Virtual C Meeting

"We want to have a proper preprocessor `#embed ...` over a `#pragma _STDC embed ...`-based directive."

This had UNANIMOUS CONSENT to pursue a proper preprocessor directive and NOT use the `#pragma` syntax. It is noted that the author deems this to be the best decision!

The following poll was later superseded in the C and C++ Committees.

"We want to specify embed as using `#embed [bits-per-element] header-name` rather than `#embed [pp-tokens-for-type] header-name`." (2-way poll.)

Y	N	A
10	2	3

— Y: 10 bits-per-element (Ye)

— N: 2 type-based (Nay)

— A: 4 Abstain (Abstain)

This poll will be a bit harder to accommodate properly. Using a *`constant-expression`* that produces a numeric constant means that the max-length specifier is now ambiguous. The syntax of the directive may need to change to accommodate further exploration.

§ 3. Introduction

For well over 40 years, people have been trying to plant data into executables for varying reasons. Whether it is to provide a base image with which to flash hardware in a hard reset, icons that get packaged with an application, or scripts that are intrinsically tied to the program at compilation time, there has always been a strong need to couple and ship binary data with an application.

Neither C nor C++ makes this easy for users to do, resulting in many individuals reaching for utilities such as `xxd`, writing python scripts, or engaging in highly platform-specific linker calls to set up `extern` variables pointing at their data. Each of these approaches come with benefits and drawbacks. For example, while working with the linker directly allows injection of very large amounts of data (5 MB and upwards), it does not allow accessing that data at any other point except runtime. Conversely, doing all of these things portably across systems and additionally maintaining the dependencies of all these resources and files in build systems both like and unlike `make` is a tedious task.

Thusly, we propose a new preprocessor directive whose sole purpose is to be `#include`, but for binary data: `#embed`.

§ 3.1. Motivation

The reason this needs a new language feature is simple: current source-level encodings of "producing binary" to the compiler are incredibly inefficient both ergonomically and mechanically. Creating a brace-delimited list of numbers in C comes with baggage in the form of how numbers and lists are formatted. C's preprocessor and the forcing of tokenization also forces an unavoidable cost to lexer and parser handling of values.

Therefore, using arrays with specific initialized values of any significant size becomes borderline impossible. One would think this old problem would be work-around-able in a succinct manner. Given how old this desire is (that comp.std.c thread is not even the oldest recorded feature request), proper solutions would have arisen. Unfortunately, that could not be farther from the truth. Even the compilers themselves suffer build time and memory usage degradation, as contributors to the LLVM compiler ran the gamut of the biggest problems that motivate this proposal in a matter of a week or two earlier this very year. Luke is not alone in his frustrations: developers all over suffer from the inability to include binary in their program quickly and perform exceptional gymnastics to get around the compiler's inability to handle these cases.

C developer progress is impeded regarding the inability to handle this use case, and it leaves both old and new programmers wanting.

Finally, Microsoft has an ABI problem with its maximum string literal size that cannot be solved using string literals or anything treated like string literals, as the LLVM thread and the thread from Claire Xen make clear. It has also frustrated both C an C++ programmers alike, despite their best efforts. It was so frustrating that even extended-C-and-C++-compilers, like Circle, solve this problem with custom directives.

§ 3.2. But *How* Expensive Is This?

Many different options as opposed to this proposal were seriously evaluated. Implementations were attempted in at least 2 production-use compilers, and more in private. To give an idea of usage and size, here are results for various compilers on a machine with the following specification:

— Intel Core i7 @ 2.60 GHz

— 24.0 GB RAM

— Debian Sid or Windows 10

— Method: Execute command hundreds of times, stare extremely hard at `htop`/Task Manager

While `time` and `Measure-Command` work well for getting accurate timing information and can be run several times in a loop to produce a good average value, tracking memory consumption without intrusive efforts was much harder and thusly relied on OS reporting with fixed-interval probes. Memory usage is therefore approximate and may not represent the actual maximum of consumed memory. All of these are using the latest compiler built from source if available, or the latest technology preview if available. Optimizations at `-O2` (GCC & Clang style)/`/O2 /Ob2` (MSVC style) or equivalent were employed to generate the final executable.

§ 3.2.1. Speed

Strategy	40 kilobytes	400 kilobytes	4 megabytes	40 megabytes
`#embed` GCC	0.236 s	0.231 s	0.300 s	1.069 s
`xxd`-generated GCC	0.406 s	2.135 s	23.567 s	225.290 s
`xxd`-generated Clang	0.366 s	1.063 s	8.309 s	83.250 s
`xxd`-generated MSVC	0.552 s	3.806 s	52.397 s	Out of Memory

§ 3.2.2. Memory Size

Strategy	40 kilobytes	400 kilobytes	4 megabytes	40 megabytes
`#embed` GCC	17.26 MB	17.96 MB	53.42 MB	341.72 MB
`xxd`-generated GCC	24.85 MB	134.34 MB	1,347.00 MB	12,622.00 MB
`xxd`-generated Clang	41.83 MB	103.76 MB	718.00 MB	7,116.00 MB
`xxd`-generated MSVC	~48.60 MB	~477.30 MB	~5,280.00 MB	Out of Memory

§ 3.2.3. Analysis

The numbers here are not reassuring that compiler developers can reduce the memory and compilation time burdens with regard to large initializer lists. Furthermore, privately owned compilers and other static analysis tools perform almost exponentially worse here, taking vastly more memory and thrashing CPUs to 100% for several minutes (to sometimes several hours if e.g. the Swap is engaged due to lack of main memory). Every compiler must always consume a certain amount of memory in a relationship directly linear to the number of tokens produced. After that, it is largely implementation-dependent what happens to the data.

The GNU Compiler Collection (GCC) uses a tree representation and has many places where it spawns extra "garbage", as its called in the various bug reports and work items from implementers. There has been a 16+ year effort on the part of GCC to reduce its memory usage and speed up initializers (C Bug Report and C++ Bug Report). Significant improvements have been made and there is plenty of room for GCC to improve here with respect to compiler and memory size. Somewhat unfortunately, one of the current changes in flight for GCC is the removal of all location information beyond the 256th initializer of large arrays in order to save on space. This technique is not viable for static analysis compilers that promise to recreate source code exactly as was written, and therefore discarding location or token information for large initializers is not a viable cross-implementation strategy.

LLVM's Clang, on the other hand, is much more optimized. They maintain a much better scaling and ratio but still suffer the pain of their token overhead and Abstract Syntax Tree representation, though to a much lesser degree than GCC. A bug report was filed but talk from two prominent LLVM/Clang developers made it clear that optimizing things any further would require an extremely large refactor of parser internals with a lot of added functionality, with potentially dubious gains. As part of this proposal, the implementation provided does attempt to do some of these optimizations, and follows some of the work done in this post to try and prove memory and file size savings. (The savings in trying to optimize parsing large array literals were "around 10%", compared to the order-of-magnitude gains from `#embed` and similar techniques).

Microsoft Visual C (MSVC) scales the worst of all the compilers, even when given the benefit of being on its native operating system. Both Clang and GCC outperform MSVC on Windows 10 or WINE as of the time of writing.

Linker tricks on all platforms perform better with time (though slower than `#embed` implementation), but force the data to be optimizer-opaque (even on the most aggressive "Link Time Optimization" or "Whole Program Optimization" modes compilers had). Linker tricks are also exceptionally non-portable: whether it is the `incbin` assembly command supported by certain compilers, specific invocations of `rc.exe`/`objcopy` or others, non-portability plagues their usefulness in writing Cross-Platform C (see Appendix for listing of techniques). This makes C decidedly unlike the "portable assembler" advertised by its proponents (and my Professors and co-workers).

§ 3.3. Support

To say that `#embed` enjoys broad C Community support is an understatement. In all the years we have written proposals for C and C++, this is the only one where someone physically mailed us a letter - from a different country - directly to the Standards Body to try and make a case for the feature directly, rather than what was already in the paper:

DMG MORI

DMG [redacted]
[redacted] Wernau
Germany
[redacted]

C Standards Committee
ISO/IEC JTC 1/SC 22/WG 14
[redacted] NV [redacted]
USA

Speaking in favor of N2592 and #embed for C23

Dear C Standards Committee,

I'm [redacted], a senior engineer at DMG MORI Japan. I write to you to speak in favor of the proposal N2592 and its possible inclusion in the C23 standard.

My relationship with the C programming language comes from graphics programming. Commanding the GPU and processing images via GLSL shaders is a joy to me in itself. Often, I embed resources in the final binary – Fonts, Icons, Shaders, etc. My Makefiles provide me a folder to drag&drop resources into, which automatically get included. Having just a single executable without the need to unpack anything and not dealing with file streams to load resources make this workflow a real joy.
Getting this workflow smoothed out across many operating systems and toolchains however, is a never-ending task and the Makefiles that make this possible have long since left the realm of readability. When multiple compilers and different platforms are involved, messy details start to surface.

For instance, with the GNU toolchain this can be done via 'ld -r -b binary -o icon_svg.o icon.svg' and linking the resulting object file to the final executable. The resource' size in the object file however, is provided by means of an absolute address. Modern compiler features like position independent executables (PIE) make the size inaccessible during run-time. The common workaround is to compute the size at run-time via 'end-pointer minus start-pointer'. A separate run-time calculation for a specific toolchain to get information already known at compile-time – this makes the code less portable. My Makefiles get the size via 'nm' and auto-generate a header file with the size information, to avoid this run-time calculation. Tiny detail in the grand scheme of things, but across many platforms, lots of work.

I look towards #embed as proposed by JeanHeyd Meneide with great anticipation. Unifying all these platform dependent details with a single #embed would make C code more portable, more comfortable and more elegant. Thus, I symbolically vote in favor of N2592 for inclusion in C23 with this letter.

Yours sincerely

[redacted]

This is just one of hundreds of messages sent over time digitally, with participants from everywhere in the European Union (France, Germany, Spain, Czech Republic, Switzerland, Denmark, etc.) to East

and Southern Asia (China, Japan, Vietnam, etc.) and many, many more from North and South America (including Canada, Brazil, Argentina, etc.). There has been a clear and present need to solve this problem for quite some time now.

§ 4. Design

There are two design goals at play here, sculpted to specifically cover industry standard practices with build systems and C programs.

The first is to enable developers to get binary content quickly and easily into their applications. This can be icons/images, scripts, tiny sound effects, hardcoded firmware binaries, and more. In order to support this use case, this feature was designed for simplicity and builds upon widespread existing practice.

The second is extensibility. We recognize that talking to arbitrary places on either the file system, network, or similar has different requirements. After feedback from an implementer about syntax for extensions, we reached out to various users of the beta builds or custom builds using `#embed`-like things. It turns out many of them have needs that, since they are the ones building and in some cases patching over/maintaining their compiler, have needs for extensible parameters that can be passed to `#embed` directives. Therefore, we structured the syntax in a way that is favorable to "simple" scanning tools but powerful enough to handle arbitrary directives and future extension points.

§ 4.1. Goal: Simplicity and Familiarity

Providing a directive that mirrors `#include` makes it natural and easy to understand and use this new directive. It accepts both chevron-delimited (`<>`) and quote-delimited (`""`) strings like `#include` does. This matches the way people have been generating files to `#include` in their programs, libraries and applications: matching the semantics here preserves the same mental model. This makes it easy to teach and use, since it follows the same principles:

```
/* default is unsigned char */
const unsigned char icon_display_data[] = {
                #embed "art.png"
};

/* specify any type which can be initialized form integer constant expressi
```

```
const char reset_blob[] = {
                #embed "data.bin"
};
```

Because of its design, it also lends itself to being usable in a wide variety of contexts and with a wide variety of vendor extensions. For example:

```
/* attributes work just as well */
const signed char aligned_data_str[] __attribute__ ((aligned (8))) = {
                #embed "attributes.xml"
};
```

The above code obeys the alignment requirements for an implementation that understands GCC directives, without needing to add special support in the `#embed` directive for it: it is just another array initializer, like everything else.

§ 4.1.1. Existing Practice - Search Paths

It follows the same implementation experience guidelines as `#include` by leaving the search paths implementation defined, with the understanding that implementations are not monsters and will generally provide `-fembed-path`/`-fembed-path=` and other related flags as their users require for their systems. This gives implementers the space they need to serve the needs of their constituency.

§ 4.1.2. Existing Practice - Discoverable and Distributable

Build systems today understand the make dependency format, typically through use of the compiler flags `-(M)MD` and friends. This sees widespread support, from CMake, Meson and Bazel to ninja and make. Even VC++ has a version of this flag -- `/showIncludes` -- that gets parsed by build systems.

This preprocessor directive fits perfectly into existing build architecture by being discoverable in the same way with the same tooling formats. It also blends perfectly with existing distributed build systems which preprocess their files with `-frewrite-includes` before sending it up to the build farm, as `distcc` and `icecc` do.

§ 4.2. Syntax

The syntax for this feature is for an extensible preprocessor directive. The general form is:

```
# embed <header-name>|"header-name" parameters...
```

where `parameters` refers to the syntax of `no_arg`/`with_arg(values, ...)`/`vendor::no_arg`/`vendor::with_arg(tokens...)` that is already part of the grammar. The syntax takes after many existing extensions in many preprocessor implementations and specifications, including OpenMP, Clang `#pragma`s, Microsoft `#pragma`s, and more. The named parameters was a recommendation by an implementer

This syntax keeps the header-name, enclosed in angle brackets or quotation marks, first to allow a "simple" preprocessing tool to quickly scan for all the necessary dependency names without having to parse any of the names or parameters that come after. Both standard names and vendor/implementation-specific names can also be accommodated in the list of parameters, allowing for specific vendor extensions in a consistent manner while the standard can take the normal `foo` names.

§ 4.2.1. Parameters

One of the things that's critical about `#embed` is that, because it works with binary resources, those resources have characteristics very much different from source and header files present in a typical filesystem. There may be need for authentication (possibly networked), permission, access, additional processing (new-line normalization), and more that can be somewhat similarly specified through the implementation-defined parameters already available through the C and C++ Standards' "`fopen`" function.

However, adding a "mode" string similar to `fopen`, while extensible, is archaic and hard to check. Therefore, the syntax allows for multiple "named parameters", encapsulated in parentheses, and marked with `::` as a form of "namespacing" identifiers similar to `[[vendor::attr]]` attribute-style syntax. However, parameters do not have the balanced square bracket `[[]]` delimiters, and just use the `vendor::attr` form with an optional parentheses-enclosed list of arguments.

Furthermore, parameters as defined in this proposal may open the door to better vendor-quality preprocessor parameters. They are defined generically and they are set to be a constraint violation (C) or make the program ill-formed (C++) if they are not recognized. This is why they are not named "attributes" and steer very far away from the attribute naming in this revision of the paper.

Some example parameters including interpreting the binary data as "text" rather than a bitstream with `clang::text(utf-8)`, providing authenticated access with `fs::auth("username", "password")`, `yosys::type(hardware_entry)` to change the element of each entry produced, and more. These are all things vendors have indicated they might support for their use cases.

4.2.1.1. Limit Parameter

The earliest adopters and testers of the implementation reported problems when trying to access POSIX-style `char` devices and pseudo-files that do not have a logical limitation. These "infinity files" served as the motivation for introducing the "limit" parameter; there are a number of resources which are logically infinite and thusly having a compiler read all of the data would result an Out of Memory error, much like with `#include` if someone did `#include "/dev/urandom"`.

The limit parameter is specified after the resource name in `#embed`, like so:

```
const int please_dont_oom_kill_me[] = {
        #embed "/dev/urandom" limit(512)
};
```

This prevents locking compilers in an infinite loop of reading from potentially limitless resources. Note the parameter is a hard upper bound, and not an exact requirement. A resource may expand to a 16-element list rather than a 512-element list, and that is entirely expected behavior. The limit is the number of elements allowed up to the maximum for this type.

This does not provide a form of "timeout" for e.g. resources stored on a Network File System or an inactivity limit or similar. Implementations that utilize support for more robust handling of resource location schemes like Uniform Resource Identifiers (URIs) that may interface with resources that take extensive amounts of time to locate should provide implementation-defined extensions for timeout or inactivity checks.

4.2.1.2. Non-Empty Prefix and Suffix

Something pointed out by others using this preprocessor directive is a problem similar to `__VA_ARGS__`: when placing this parameter with other tokens before or after the `#embed` directive, it sometimes made it hard to properly anticipate whether a file was empty or not.

The `#embed` proposal includes a prefix and suffix entry that applies if and only if the resource is non-empty:

```
const unsigned char null_terminated_file_data[] = {
        #embed "might_be_empty.txt" \
                prefix(0xEF, 0xBB, 0xBF, ) /* UTF-8 BOM */ \
                suffix(,)
        0 // always null-terminated
};
```

`prefix` and `suffix` only work if the `#embed` resource is not empty. If a user wants a prefix or suffix that appears unconditionally, they can simply just type the tokens they want before and after: there is nothing to be gained from adding a standards-mandated prefix and suffix that works in both the empty and non-empty case.

We do not want to entirely lose that user's use case, however, so we have made the `suffix`/`prefix` parameters an **optional** part of the wording, to be voted on as a **separate** piece.

§ *4.2.1.3. Empty Signifier*

This is for the case when the given resource exists, but it is empty. This allows a user to have a sequence of tokens between the parentheses passed to the `if_empty` parameter here: `#embed "blah" if_empty(SPECIAL_EMPTY_MARKER MORE TOKENS)`.

If `"blah"` exists but is empty, this will replace the directive with the (potentially macro expanded) contents between the parentheses of the `if_empty` parameter. This can also be combined with a `limit(0)` parameter to always have the `if_empty` token return. This can be useful for macro-expanded integer constant expressions that may end up being 0.

An example program `single-urandom.c`:

```
int main () {
#define SOME_CONSTANT 0
    return
#embed </dev/urandom> if_empty(0) limit(SOME_CONSTANT)
    ;
}
```

This program will expand to the equivalent of `int main () { return 0; }` if `SOME_CONSTANT` is 0, or a single (random) `unsigned char` value if it is 1. (If `SOME_CONSTANT` is greater than 1, it produces a comma-delimited list of integers, which gets treated as a sequence to the comma operator after the `return` keyword. Some compilers warn about the left-hand operands having no effect.)

Previously, this was the only way to detect that the resource was empty. This functionality can be substituted with having to use `__has_embed(…)` with the same contents and specifically check for the return value of `== 2`. While this change create some repeating-yourself friction in the identifier, there was only 1 user who actually needed the if_empty signifier, and that was only because they were using it to replace it with a very particularly sized and shaped data array. The `__has_embed` technique worked just fine for them as well at the cost of some repetition (to check for embed parameters), and after some discussion with the user it was deemed okay to switch to this syntax, since during the discussion of `#embed` in the January/February 2022 WG14 C Standards Committee Meeting it was commented on that there were too many signifiers.

We do not want to entirely lose that user's use case, however, so we have made the `if_empty` parameter an **optional** part of the wording, to be voted on as a **separate** piece.

4.2.1.4. Why is `__param_name__` part of the grammar?

This is specifically at the request of the C Committee. There are plenty of places where `limit`/`suffix`/`prefix` may be preexisting macro names. Thus, following the same rationale of other preprocessor-hardened names (such as attributes in the C standard), names that are prefixed and suffixed by a double underscore (e.g., `__limit__`, `__suffix__`, and similar) behave identically to their non-underscore counterparts. This is to aid in differentiation to prevent macro collisions, and is featured as part of the style of attributes in C++.

This feature will only be part of the C version of the proposal, as C++ does not have a similar rule for its attributes and therefore has no precedent amongst C++ compilers. (We expect that dual-C-and-C++ compilers will support both spellings in the interest of ease-of-shared-implementation.)

4.3. Constant Expressions

Both C and C++ compilers have rich constant folding capabilities. While C compilers only acknowledge a fraction of what is possible by larger implementations like MSVC, Clang, and GCC, C++ has an entire built-in compile-time programming bit, called `constexpr`. Most typical solutions cannot be used as constant expressions because they are hidden behind run-time or link-time mechanisms (`objcopy`, or the resource compiler `rc.exe` on Windows, or the static library archiving

tools). This means that many algorithms and data components which could strongly benefit from having direct access to the values of the integer constants do not because the compiler cannot "see" the data, or because Whole Program Optimization cannot be aggressive enough to do anything with those values at that point in the compilation (i.e., during the final linking stage).

This makes `#embed` especially powerful, since it guarantees these values are available as-if it was written by as a sequence of integers whose values fit within an `unsigned char`.

§ 4.4. `__has_embed`

C and C++ both support a `__has_include` . It makes sense to have an analogous `__has_embed` identifier. It can take a `__has_embed( "header-name" ... )` or `__has_embed (<header-name> ... )` resource name identifier, as well as additional arguments to let vendors pass in any additional arguments they need to properly access the file (following the same parameters passed to the directive). `__has_embed` evaluates to:

— `0` if the resource is not found or any parameter in the `embed-parameter-list` does not exist; or,

— `1` if the resource is found, it is not empty, and the `embed-parameter-list` (including the vendor-specific ones) are supported; or,

— `2` if the resource is found, it is empty, and the `embed-parameter-list` (including the vendor-specific ones) are supported.

This may raise questions of "TOCTTOU" (Time of Check to Time of Use) problems, but we already have these problems between `__has_include` and `#include`. They are also already solved by existing implementations. For example, the LLVM/Clang compiler uses `FileManager` and `SourceManager` abstractions which cache files. GCC's "libcpp" (not its C++ library but it's preprocessor library) will cache already-opened files (up to a limit). Any TOCTTOU problems have already been managed and provided for using the current `#include` infrastructure of these compilers, and if any compiler wants a more streamlined and consistent experience they should deploy whatever Quality of Implementation (QoI) they see fit to achieve that goal.

Finally, note that this directive DOES expand to `0` if a given parameters that the implementation does not support. This makes it easier to determine if a given vendor-specific embed directive is supported.

In fact, support can be checked in most cases by using a combination of `__FILE__` and `__has_embed`:

```
int main () {
#if __has_embed ("bits.bin" clang::element_type(short))
        // load "short" values directly from memory
        short meow[] = {
#embed "bits.bin" clang::element_type(short)
        };
#else
        // no support for implementation-specifid
        // clang::element_type parameter
        unsigned char meow_bytes[] = {
#embed "bits.bin"
        };
        unsigned short meow[] = {
                /* parse meow_bytes into short values
                   by-hand! */
        };
#endif
        return 0;
}
```

For the C proposal, the wording for `__has_embed(...)` returning `2` is optional, as it depends on whether or not the C Committee would like to solve this problem in one specific direction or another.

§ 4.5. Bit Blasting: Endianness

> What would happen if you did `fread` into an `int`?
>
> that's my answer 🙂
>
> – Isabella Muerte

It's a simple answer. While we may not be reading into `int`, the idea here is that the interpretation of the directive is meant to get as close to directly copying the bitstream, as is possible. A compiler-magic based implementation like the ones provided as part of this paper have no endianness issues, but an implementation which writes out integer literals may need to be careful of host vs. target endianness to

make sure it serializes correctly to the final binary. As a litmus test, the following code -- given a suitably sized `"foo.bin"` resource -- should return `0`:

```
#include <cstdio>
#include <cstring>

int main() {
        const unsigned char foo0[] = {
#embed "foo.bin"
        };

        const unsigned char foo1[sizeof(foo0)];
        std::FILE* fp = std::fopen("foo.bin");
        if (fp == nullptr) {
                return 1;
        }
        std::size_t foo1_read = std::fread(foo1, 1, sizeof(foo1), fp);
        if (foo1_read != sizeof(foo1)) {
                return 1;
        }
        if (memcmp(&foo0[0], &foo1[0], sizeof(foo0)) != 0) {
                return 1;
        }
        return 0;
}
```

If the same file during both translation and execution, `"foo.bin"`, is used here, this program should always return `0`. This is what the wording below attempts to achieve. Note that this is always a concern already, due to `CHAR_BIT` and other target environment-specific variables that already exist; implementations have always been responsible for handling differences between the host and the target and this directive is no different. If the `CHAR_BIT` of the host vs. the target is the same, then the directive is more simple. If it is not, then an implementation will have to perform translation.

5. Implementation Experience

An implementation of this functionality is available in branches of both GCC and Clang, accessible right now with an internet connection through the online utility Compiler Explorer. The Clang compiler with this functionality is called "x86-64 clang (thephd.dev)" in the Compiler Explorer UI:

```
int main () {
    return
#embed </dev/urandom> limit(1)
    ;
}
```

6. Alternative Syntax

There were previous concerns about the syntax using pragma-like syntax and more. WG14 voted to keep the syntax as a plain `#embed` preprocessor directive, unanimously.

Previously, different syntax was used to specify the limit and other kinds of parameters. These have been normalized to be a suffix of attribute-like parameters, at the request of an implementer and the C++ Standards Committee discussion of the paper in June 2021. It has had hugely positive feedback and users have reported the new syntax to be clearer, while other implementers have stated this is much better for them and the platforms for which they intend to add additional embed parameters.

6.1. `__has_embed(…) == 2`, `suffix`/`prefix`/`if_empty` parameters, both, or neither?

This proposal contains two different ways to handle empty parameters. Both are optionally included in this proposal to be voted on by WG21 (C++) and WG14 (C) respectively. While `__has_embed` can be used specifically to check if a resource is empty (by checking if it expands to a value of `2`), `prefix`, `suffix`, and `if_empty` reduce the complexity of the series of necessary checks in order to handle specific situations. For example, let's take a common situation of checking if a resource is empty, and if it isn't simply defaulting some data to be empty:

```
static_assert(CHAR_BIT == 8, "expects an 8-bit char.");

const unsigned char raw_data[] = {
#if __has_embed(<some_file.txt>) == 2
        // file is empty
```

```
#else
        // file has content, or implementation defined search fails (errors
#       embed <some_file.txt>
        , // need the extra ',' in this case
#endif
        0, 0
};
```

Now, let's add checks for platform-specific-data, using just `__has_embed` as before:

```
static_assert(CHAR_BIT == 8, "expects an 8-bit char.");

const unsigned char raw_data[] = {
#if __has_embed(<some_file.txt> acme::open_mode("x,nt")) == 1
        // supports the directive: very nice!
#       embed <some_file.txt> acme::open_mode("x,nt")
#elif __has_embed(<some_file.txt>) == 2
        // file is empty: nothing needed
#else
        // file has content, or implementation defined search fails (errors
#       embed <some_file.txt>
        , // need the extra ',' in this case
#endif
        0, 0
};
```

Each new condition adds yet another branch. In the case of something like a `clang::element_type(type)` which supports doing "direct binary translation into a sequence of objects with type `type`" or similar, this can become a very lengthy list of supported directives. This does not necessarily mean a world with the empty-handling parameters is more helpful: `__has_embed` is still needed to check whether implementation-defined parameters are allowed:

```
static_assert(CHAR_BIT == 8, "expects an 8-bit char.");

const unsigned char raw_data[] = {
#if __has_embed(<some_file.txt> acme::open_mode("x,nt"))
        // supports the directive: very nice!
#       embed <some_file.txt> acme::open_mode("x,nt")
#else
        // file has content, is empty, or implementation defined
```

```
        // search fails (errors) suffix takes care of adding the
        // necessary comma if it's empty
#       embed <some_file.txt> suffix(,)
        0, 0
#endif
};
```

We only lose a single `#elif` branch here. For more involved or complicated code that cares even more deeply about adding prefixes (UTF-8 prefixes) or different kinds of suffixes / empty-handlers, the reduction for handling empty cases may drop much more. For the case where we're not worried about special vendor extensions at all, the code is shorter:

```
static_assert(CHAR_BIT == 8, "expects an 8-bit char.");

const unsigned char raw_data[] = {
        // file has content, or implementation defined search fails (errors
        // suffix takes care of adding the necessary comma
#embed <some_file.txt> suffix(,)
        0, 0 // need the extra ',' in this case
};
```

We do not actually have a terrible preference about how much we would like to force users to make the ladder of `#if`/`#elif`. We introduced the parameter-based approach for empty files before we introduced the tri-state `__has_embed(…)` replacement value in the preprocessor. Users found the fix for empty files targeted and helpful, but perhaps that is not the best "general purpose" solution. The "general purpose" solution does make doing tests potentially complex, but maybe that complexity results in an overall more pleasant and coherent experience.

We leave it up to the Committee to pick one, the other, neither, or both of the given ways of handling empty files (and, importantly, staving off errors from empty contiguous sequences, C-style arrays in particular).

§ 6.2. Why a Preprocessor Directive, Specifically?

Although the reasoning is scattered around the paper, it may be illuminating to explain the full "etymology" of the preprocessor directive of `#embed` and why it came to be. Originally, `#embed` was conceived not by the authors of this paper, but as a C++ feature (potentially portable to C) using a String Literal of the form `F"file.ext"` or `bF"file.ext"`. This proposal was soundly rejected by

WG21, despite having one of the strongest champions it could possibly have presenting it (the original author of the paper could not make Committee Meetings). There was much confusion about whether or not this functionality would include a null terminator (some argued "yes", because it was the form of a string literal; others argued "no"). There was also the confusion between what it meant to include the file as a "text" file versus a "binary" file (the `F""` versus `bF""` prefixes). Furthermore, there was the question of what kind of data would come out on the other side (`char` for "text" data, `unsigned char` for binary?).

From there, the authors of this paper worked to explore a new version that was, effectively, language magic. This was done because, very long ago, solutions around doing `#include_binary` or similar were (colloquially) rejected from both WG21 and WG14, with various different reasons. This is where the C++-shaped `std::embed("file.ext")` came from, and forms the basis of the C++ proposal [p1040r6]. It was meant to be filled in using either directly compiler-based language magic such as a built-in (like `__builtin_embed`), or using compiler-specific extensions such as `.incbin` ([incbin]) but mightily improved to be usable as a constant expression. This approach found great traction until tool vendors and tool developers within compiler groups complained that such a construct - especially in C++ - would allow evaluation of more than just String Literals as the entry into `std::embed`. This is partly a feature, as it meant that reading a file and pulling file names from it could reuse the included data to then feed it back into the magic directive to read another file, resulting in the ability to parse e.g. GLSL shader files or JSON files which referenced other JSON files into read-only, compiled memory. Unfortunately, such ability means that it is beyond the Phase 1-5, preprocessor abilities of C and C++ compilers, and dependencies must be computed at what is typically known as "Semantic Analysis" time in compilers.

This also proved problematic for tool developers outside of the C++ Standards Committee. Correspondence with Henry Miller of the `icecc` (distributed build tool) development list over e-mail revealed that while he would be comfortable just having a loosely-based "best effort" approach to finding file names from any `std::embed("...")` function call. He indicated some preference for a more static version that could allow him to find all potentially-included files without any risk of false-positives or other issues, that a simple tool like `icecc` could handle.

This is where `#embed` was conceived. While it loses the ability to be used recursively with itself like the language-magic version in C++, it retains the following important qualities for all stakeholders involved:

— the directive can be read using basic preprocessing tools;

— the directive has parameters that come after the core `#embed "file.txt"` part, which means current infrastructure around parsing similar `#include` files is retained;

— the directive does not need any information or computation from beyond Phase 4 of compilation; and,

— the directive does not have any questions of "encoding" or "null termination" like the File String Literals proposal.

This is the version that stayed in development since around late 2018, and has been continuously iterated over. The syntax was changed once, to accommodate the trailing parameter list, as suggested by both compiler developers and end-users several times during the course of its development. The trailing parameter list was also the most powerful way to allow compiler extensions that did special behaviors, as it was incredibly clear that many vendors had extensions for data, attributes for variables, and more they wanted to feed into `#embed` and the various types it initialized. It took some time to smith the wording into the form that works best, but importantly the wording from the beginning had the following two goals in mind:

— a low-effort quality of implementation would still work validly in all places it could conceivably be used (which is anywhere, since it's a preprocessor directive); and,

— a high-effort quality of implementation could significantly speed up the inclusion of binary resources in a program.

Notably, the first was important for users. They did not want something that would only be conditionally supported. The latter is important for both users and vendors: Microsoft needs a way to get out of its String Literal Maximum Limit ABI problems and a way to store data quickly, as every other compiler even on its preferred platform (Windows 7, 8, 8.1, and 10) are outperformed by other compilers simply initializing data arrays in a big, brace-delimited array. In order to solve for the bug reports talked about earlier ([nonius-visual-c-error], [llvm-string-init-fail], [gcc-large-init-bug-c], and more) vendors need the capability to provide a fast built-in or internal token implementation for speed purposes. This has been validated by implementation efforts outside of the authors here, in the QAC Compiler by Alex Gilding. With the authors currently privately and publicly supporting implementations of `#embed`, Sean Baxter's own implementation of embed styled slightly differently for the Circle compiler, and the implementation work by Alex, that brings us to 4 compilers (Clang, GCC, Circle, QAC) - private and commercially available - with experience and reported order-of-magnitude (and in some cases, two orders-of-magnitude) improvements on compilation and loading speed of data arrays previously stored in a variety of other manners, including linkers.

It is a long-battled, well-storied work of shared and community effort to solve a long-standing problem for C and C++. It gives vendors a way out of having to write increasingly integer-list-initialization-specific optimized parsers, and lets users connect C to the best binary and storage compression system they already know how to use with their existing implementations: the filesystem. It also prevents low-effort quality of implementations from being excluded from the feature set, making it feasible even for compilers such as the famous 8cc hobby compiler.

This is why `#embed` is in the form it has ultimately ended up in.

7. Wording

This wording is relative to C++'s latest working draft.

7.1. Intent

The intent of the wording is to provide a preprocessing directive that:

— takes a quote or chevron delimited header-name -- potentially from the expansion of a macro -- and uses it to find a unique resource in an implementation-defined manner;

— behaves as if it produces a list of values suitable for the initialization of an array as well as initializes each `unsigned char` element according to the specific environment limits found in an implementation-defined manner;

— errors if the size of the resource does not have enough bits to fully and properly initialize all the values generated by the directive;

— allows a limit parameter limiting the number of elements to be specified (but allowing less than the limit);

— produces a core constant expression that can be used to initialize `constexpr` arrays;

— and, present such contents as if it is a list of values, such that it can be used to initialize arrays of known and unknown bound even if additional elements of the whole initialization list come before or after the directive.

7.2. Proposed Feature Test Macro

The proposed feature test macro is `__cpp_pp_embed` for the preprocessor functionality.

7.3. Proposed Language Wording

7.3.1. Append to §14.8.1 Predefined macro names [cpp.predefined] an additional entry

```
#define __cpp_pp_embed     ????? /* 📝 NOTE: EDITOR VALUE HERE */
```

§ **7.3.2. Add to the *control-line* production in §15.1 Preamble [cpp.pre] a new grammar production, as well as a supporting *embed-parameter-seq* production**

embed-parameter:

embed-standard-parameter

embed-prefixed-parameter

embed-standard-parameter:

limit **(** *constant-expression* **)**

embed-prefixed-parameter:

identifier **::** *identifier embed-parameter-clause*

embed-parameter-clause:

(*pp-balanced-token-seq*$_{opt}$ **)**

pp-balanced-token-seq:

pp-balanced-token

pp-balanced-token-seq pp-balanced-token

pp-balanced-token:

(*pp-balanced-token-seq*$_{opt}$ **)**

[*pp-balanced-token-seq*$_{opt}$ **]**

{ *pp-balanced-token-seq*$_{opt}$ **}**

any *pp-token* other than a parenthesis, a bracket, or a brace

embed-parameter-seq:

embed-parameter

embed-parameter-seq embed-parameter

control-line:

...

> # `embed` *pp-tokens new-line*
>
> …
>
> 8✨ Any *embed-prefixed-parameter* is conditionally-supported, with implementation-defined semantics.

§ **7.3.3. Modify §15.2 Conditional inclusion [cpp.cond] to include a new "has-embed-expression" by modifying paragraph 1 and adding a new paragraph 5 after the current paragraph 4**

> *has-embed-expression*:
>
> ...
>
> `__has_embed (` *header-name embed-parameter-seq*$_{opt}$ `)`
>
> `__has_embed (` *header-name-tokens pp-balanced-token-seq*$_{opt}$ `)`

1 … and it may contain zero or more *defined-macro-expressions*, *has-include-expressions*, ~~and/or~~ *has-attribute-expressions*, and/or *has-embed-expressions* as unary operator expressions.

…

3 The second form of either the *has-include-expression* or the *has-embed-expression* is considered only if the first form does not match, in which case the preprocessing tokens are processed just as in normal text.

…

5 The resource identified by the *header-name* in each contained *has-embed-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a `# embed` directive ([cpp.res]). If such a directive would not satisfy the syntactic requirements of a `#embed` directive, the program is ill-formed. The *has-embed-expression* evaluates to:

(5.1) — `0` if the search fails or any given embed parameters in the *embed-parameter-seq* are not supported.

(5.2) — Otherwise, `1` if the search for the resource succeeds, all the given embed parameters in the *embed-parameter-seq* are supported.

[*Note:* An unrecognized embed parameter given to a *has-embed-expression* is not ill-formed. — *end note*]

§ **7.3.4. Add a new sub-clause §15.4 Resource inclusion [cpp.res]**

15.4 Resource inclusion [cpp.res]

15.4.1 General [cpp.res.gen]

1 A preprocessing directive of the form

`# embed` < *h-char-sequence* > *embed-parameter-seq*$_{opt}$ *new-line*

searches a sequence of implementation-defined places for a resource identified uniquely by the specified sequence between the < and > delimiters, and causes the replacement of that directive by a comma-delimited list of integer constant expressions as specified below. It is implementation-defined how the places or how the resource is identified.

Recommended Practice: A mechanism similar to, but distinct from, the implementation-defined search paths used for ([cpp.include]) is encouraged.

2 A preprocessing directive of the form

`# embed "` *q-char-sequence* `"` *embed-parameter-seq*$_{opt}$ *new-line*

searches a sequence of implementation-defined places for a resource identified uniquely by the specified sequence between the " and " delimiters. It is implementation-defined how the places or how the resource is identified.

If this search is not supported, or if the search fails, the directive is reprocessed as if it read

`# embed` < *h-char-sequence* > *embed-parameter-seq*$_{opt}$ *new-line*

with the identical contained sequence (including " characters, if any) from the original directive.

Recommended Practice: A mechanism similar to, but distinct from, the implementation-defined search paths used for ([cpp.include]) is encouraged.

3 An `#embed` directive shall identify a resource that can be processed into a comma-delimited list of integer literals. A *resource* is a source of data accessible from the translation environment. An `# embed` directive can take an arbitrary number of embed parameters after the *q-char-sequence* or *h-char-sequence*, separated by white space. This is the *embed-parameter-seq*, described in ([cpp.res.param]). An *embed-parameter* is a single embed parameter in the *embed-parameter-seq*. A resource has an *implementation-resource-width*,

which is the implementation-defined size in bits of the located resource. It also has a *resource-width*, which is:

(3.1) — the number of bits as computed from the optionally-provided `limit` *embed-parameter* ([cpp.res.param.limit]), if present.

(3.2) — Otherwise, the implementation-resource-width.

4 Let *embed-element-width* be:

(4.1) — a value greater than zero determined by an implementation-defined embed parameter, if present.

(4.2) — Otherwise, `CHAR_BIT`.

The resource-width shall be an integral multiple of the embed-element-width.

[*Example*:

```
int main (int, char*[]) {
        const unsigned char coeffs[] = {
// ill-formed if the resource-width is 6 bits and
// the embed-element-width is CHAR_BIT (which is, at minimum, 8 bits
#embed "6_bits.bin"
        };

        const unsigned char fac[] = {
// may be ill-formed:
// (resource-width) % (embed-element-width)
// may not be 0 on an implementation where the resource-width is 12
#embed "12_bits.bin"
        };

        return 0;
}
```

– end example]

5 Either form of the `# embed` directive expands to a comma-separated list of integer literals, where each integer literal is `static_cast` ([expr.static.cast]) to `unsigned char`. Each integer literal shall have an implementation-defined value between `0` and `2^embed-element-width - 1`, inclusive. If that list is used to initialize a contiguous sequence of `unsigned char`, or `char` if it is not capable of representing negative values, the elements of the

sequence from the comma-separated list shall contain values as-if read by `std::fread` ([library.c]) from the resource as a file during program execution.

Recommended Practice: Each integer literal produced should closely represent the bit stream of the resource unmodified. This may require an implementation to consider potential differences between translation and execution environments, endianness, and any other applicable sources of mismatch.

[*Example*:

```
#include <cstring>
#include <cstddef>
#include <fstream>
#include <vector>

int main() {
	const unsigned char d[] = {
#embed <data.dat>
	};
	const std::vector<unsigned char> vec_d = {
#embed <data.dat>
	};

	constexpr std::size_t expected_size = sizeof(d);

	// same file in execution environment
	// as was embedded
	std::ifstream f_source("data.dat", std::ios::binary | std::i
	unsigned char runtime_d[expected_size];
	char* ifstream_ptr = reinterpret_cast<char*>(runtime_d);
	if (!f_source.read(ifstream_ptr, expected_size))
		return 1;

	std::size_t ifstream_size = f_source.gcount();
	if (ifstream_size != expected_size)
		return 2;

	int is_same = std::memcmp(&d[0], ifstream_ptr, ifstream_size
	if (is_same != 0)
		return 3;

	int is_same_vec = std::memcmp(vec_d.data(), ifstream_ptr, if
	if (is_same_vec != 0)
```

```
                return 4;

        // if the file is the same as the one in the translation env
        // the program reaches this statement and returns 0
        return 0;
}
```

— end example]

6 [*Example*:

```
#include <cstddef>

void have_you_any_wool(const unsigned char*, std::size_t);

int main (int, char*[]) {
        constexpr const unsigned char baa_baa[] = {
#embed "black_sheep.ico"
        };

        have_you_any_wool(baa_baa, sizeof(baa_baa));

        return 0;
}
```

– end example]

7 A preprocessing directive of the form

`#` `embed` *pp-tokens new-line*

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after `embed` in the directive are processed just as in normal text. (Each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens.) The directive resulting after all replacements shall match one of the two previous forms [Note: Adjacent string literals are not concatenated into a single string literal; thus, an expansion that results in two string literals is an invalid directive. — end Note]. The method by which a sequence of preprocessing tokens between a `<` and a `>` preprocessing token pair or a pair of `"` characters is combined into a single resource name preprocessing token is implementation-defined.

[*Example*:

```
#define INT_DATA_H "i.dat"

int main () {
        int i = {
#embed INT_DATA_H
        }; // well-formed if i.dat produces 1 value
        struct s {
                double a, b, c;
                struct { double e, f, g; };
                double h, i, j;
        };
        s x = {
        // well-formed, initializes each element in
        // order according to initialization rules for a
        // brace-delimited, comma-separated list
#embed "s.dat"
        };
        return 0;
}
```

– end example]

§ **7.3.5. Add a new sub-clause §15.4.2 under Resource Inclusion for Embed parameters [cpp.res.param]**

15.4.2 Embed parameters [cpp.res.param]

15.4.2.1 General [cpp.res.param.gen]

1 The *embed-parameter-seq* contains embed parameters separated by whitespace. The use of embed parameter may modify the result of the replacement for a `# embed` preprocessing directive.

15.4.2.2 `limit` parameter [cpp.res.param.limit]

1 An *embed-parameter* of the form `limit ( constant-expression )` denotes a maximum number of elements that may be produced in the comma-delimited list. It may appear zero or one times in the *embed-parameter-seq*.

2 The *constant-expression* shall be processed as described in conditional inclusion ([cpp.cond]). It shall be an integral constant expression greater than or equal to zero after evaluation. The token `defined` shall not appear in the *constant-expression*. The *constant-expression* tokens are processed just as in normal text before the evaluation of the integer constant expression.

3 The resource-width is:

(3.1) — `0`, if the integer constant expression evaluates to `0`.

(3.2) — Otherwise, the implementation-resource-width, if it is less than the embed-element-width multiplied by the integer constant expression.

(3.3) — Otherwise, the embed-element-width multiplied by the integer constant expression, if it is less than or equal to the implementation-resource-width.

[*Example*:

```
#include <cassert>

int main (int, char*[]) {
        constexpr const char sound_signature[] = {
#embed <sdk/jump.wav> limit(2+2)
        };

        // verify PCM WAV resource
```

```
        assert(sound_signature[0] == 'R');
        assert(sound_signature[1] == 'I');
        assert(sound_signature[2] == 'F');
        assert(sound_signature[3] == 'F');
        assert(sizeof(sound_signature) == 4);

        return 0;
}
```

– end example]

[*Example*:

```
int main (int, char*[]) {
        const unsigned char scal[] = {
// may be ill-formed: if resource-width is greater than 24,
// or if ((resource-width * 1) % 24) is not equivalent to 0
#embed "24_bits.bin" limit(1)
        };

        return 0;
}
```

– end example]

§ 7.3.6. OPTIONAL: Modify the previously-added clause for `__has_embed` to return `2` if the list of supported files is empty and add example

…

5 The resource identified by the *header-name* in each contained *has-embed-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a `# embed` directive ([cpp.res]). If such a directive would not satisfy the syntactic requirements of a `#embed` directive, the program is ill-formed. The *has-embed-expression* evaluates to:

(5.1) — `0` if the search fails or any given embed parameters in the *embed-parameter-seq* are not supported.

(5.2) — Otherwise, `1` if the search for the resource succeeds, all the given embed parameters in the *embed-parameter-seq* are supported , and the resource is not empty .

(5.3) — Otherwise, `2` if the search for the resource succeeds, all the given embed parameters in the *embed-parameter-seq* are supported, and the resource is empty.

...

§ 7.3.7. OPTIONAL: Add a new sub-clause §15.4.2.3-5 under Resource Inclusion for Embed parameters `prefix`, `suffix`, and `if_empty` [cpp.res.param.prefix/suffix/if_empty]

embed-standard-parameter:

limit **(** *constant-expression* **)**

prefix **(** *pp-balanced-token-seq*$_{opt}$ **)**

suffix **(** *pp-balanced-token-seq*$_{opt}$ **)**

if_empty **(** *pp-balanced-token-seq*$_{opt}$ **)**

15.4.2.3 `prefix` parameter [cpp.res.param.prefix]

1 An *embed-parameter* of the form `prefix ( pp-balanced-token-seq`$_{opt}$`)` denotes a sequence of tokens to be placed before the expansion of the `# embed` directive, if the resource is not empty. It may appear zero or one times in the *embed-parameter-seq*.

2 If the resource is empty, this *embed-parameter* is ignored. Otherwise, any tokens from the *pp-balanced-token-seq* shall be placed immediately before the expansion of the `# embed` directive.

15.4.2.4 `suffix` parameter [cpp.res.param.suffix]

1 An *embed-parameter* of the form `suffix ( pp-balanced-token-seq`$_{opt}$`)` denotes a sequence of tokens to appear after the expansion of the `# embed` directive, if the resource is not empty. It may appear zero or one times in the *embed-parameter-seq*.

2 If the resource is empty, this *embed-parameter* is ignored. Otherwise, any tokens from the *pp-balanced-token-seq* shall be placed directly after the expansion of the `# embed` directive.

[*Example*:

```
#include <cstring>
#include <cassert>

#ifndef SHADER_TARGET
#define SHADER_TARGET "ches.glsl"
#endif

extern char* merp;

void init_data () {
        constexpr const char whl[] = {
#embed SHADER_TARGET \
                prefix(0xEF, 0xBB, 0xBF, ) /* UTF-8 BOM */ \
                suffix(,)
                0
        };
        // always null terminated,
        // contains BOM if not-empty
        bool is_good = (sizeof(whl) == 1 && whl[0] == '\0')
        || (whl[0] == '0xEF' && whl[1] == '0xBB'
            && whl[2] == '0xBF' && whl[sizeof(whl) - 1] == '\0');
```

```
        assert(is_good);
        std::strcpy(merp, whl);
}
```

– end example]

15.4.2.5 `if_empty` parameter [cpp.res.param.if_empty]

1 An *embed-parameter* of the form `if_empty ( pp-balanced-token-seq`$_{opt}$ `)` denotes a sequence of tokens to use if the resource is empty. It may appear zero or one times in the *embed-parameter-seq*.

2 If the resource is not empty, this *embed-parameter* is ignored. Otherwise, any tokens from the *pp-balanced-token-seq* shall be placed where the embed directive is.

[*Example*: If some resource `"empty_file.dat"` is empty, then this

```
constexpr const char x[] = {
#embed "empty_file.dat" \
        if_empty((char)-1)
};
```

expands to

```
constexpr const char x[] = {
        (char)-1
};
```

Otherwise, it expands to the contents of the resource. *– end example*]

[*Example*: `limit(0)` affects when a file is considered empty. Therefore, the following program:

```
constexpr const char x[] = {
#embed "very_large_file.dat" \
        if_empty((char)-1) limit(0)
};
```

expands to

```
constexpr const char x[] = {
        (char)-1
};
```

> Otherwise, it expands to the contents of the resource. – *end example*]

§ *7.3.7.1. 📝 If the `if_empty` parameter is accepted with the `__has_embed` changes, add this example to the end of the `if_empty` section:*

> **15.4.2.5 `if_empty` parameter [cpp.res.param.if_empty]**
>
> …
>
> 2 …
>
> [*Example*: This resource is considered empty due to the `limit(0)` *embed-parameter*, always, including in `__has_embed` clauses.
>
> ```
> int main () {
> #if __has_embed(</owo/uwurandom> limit(0) prefix(some tokens)) == 2
>         // if </owo/uwurandom> exits, this
>         // conditional inclusion branch is taken and the program
>         // returns 0.
>         return 0;
> #else
>         // the resource does not exist
>         #error "The resource does not exist"
> #endif
> }
> ```
>
> – *end example*]

§ 8. Acknowledgements

Thank you to Alex Gilding for bolstering this proposal with additional ideas and motivation. Thank you to Aaron Ballman, David Keaton, and Rajan Bhakta for early feedback on this proposal. Thank you to the `#include<C++>` for bouncing lots of ideas off the idea in their Discord. Thank you to Hubert Tong for refining the proposal's implementation-defined extension points.

Thank you to the Lounge<C++> for their continued support, and to rmf for the valuable early implementation feedback.

9. Appendix

9.1. Existing Tools

This section categorizes some of the platform-specific techniques used to work with C++ and some of the challenges they face. Other techniques used include pre-processing data, link-time based tooling, and assembly-time runtime loading. They are detailed below, for a complete picture of today's landscape of options. They include both C and C++ options.

9.1.1. Pre-Processing Tools

1. Run the tool over the data (`xxd -i xxd_data.bin > xxd_data.h`) to obtain the generated file (`xxd_data.h`) and add a null terminator if necessary:

```
unsigned char xxd_data_bin[] = {
        0x48, 0x65, 0x6c, 0x6c, 0x6f, 0x2c, 0x20, 0x57, 0x6f, 0x72, 0x6c, 0
        0x0a, 0x00
};
unsigned int xxd_data_bin_len = 13;
```

2. Compile `main.c`:

```
#include <stdlib.h>
#include <stdio.h>

// prefix as const,
// even if it generates some warnings in g++/clang++
const
#include "xxd_data.h"

int main() {
                const char* data = reinterpret_cast<const char*>(xxd_data_b
                puts(data); // Hello, World!
                return 0;
}
```

Others still use python or other small scripting languages as part of their build process, outputting data in the exact C++ format that they require.

There are problems with the `xxd -i` or similar tool-based approach. Tokenization and Parsing data-as-source-code adds an enormous overhead to actually reading and making that data available.

Binary data as C(++) arrays provide the overhead of having to comma-delimit every single byte present, it also requires that the compiler verify every entry in that array is a valid literal or entry according to the C++ language.

This scales poorly with larger files, and build times suffer for any non-trivial binary file, especially when it scales into Megabytes in size (e.g., firmware and similar).

9.1.2. `python`

Other companies are forced to create their own ad-hoc tools to embed data and files into their C++ code. MongoDB uses a custom python script, just to format their data for compiler consumption:

```
import os
import sys

def jsToHeader(target, source):
                outFile = target
                h = [
                                '#include "mongo/base/string_data.h"',
                                '#include "mongo/scripting/engine.h"',
                                'namespace mongo {',
                                'namespace JSFiles{',
                ]
                def lineToChars(s):
                                return ','.join(str(ord(c)) for c in (s.rst
                for s in source:
                                filename = str(s)
                                objname = os.path.split(filename)[1].split(
                                stringname = '_jscode_raw_' + objname

                                h.append('constexpr char ' + stringname + '

                                with open(filename, 'r') as f:
                                                for line in f:
                                                                h.append(li

                                h.append("0};")
                                # symbols aren't exported w/o this
```

```
                                h.append('extern const JSFile %s;' % objnam
                                h.append('const JSFile %s = { "%s", StringD
                                                                (objname,

                    h.append("} // namespace JSFiles")
                    h.append("} // namespace mongo")
                    h.append("")

                    text = '\n'.join(h)

                    with open(outFile, 'wb') as out:
                                try:
                                                out.write(text)
                                finally:
                                                out.close()


if __name__ == "__main__":
                    if len(sys.argv) < 3:
                                print "Must specify [target] [source] "
                                sys.exit(1)
                    jsToHeader(sys.argv[1], sys.argv[2:])
```

MongoDB were brave enough to share their code with me and make public the things they have to do: other companies have shared many similar concerns, but do not have the same bravery. We thank MongoDB for sharing.

9.1.3. `ld`

A complete example (does not compile on Visual C++):

1. Have a file ld_data.bin with the contents `Hello, World!`.
2. Run `ld -r binary -o ld_data.o ld_data.bin`.
3. Compile the following `main.cpp` with `gcc -std=c++17 ld_data.o main.cpp`:

```
#include <stdlib.h>
#include <stdio.h>

#define STRINGIZE_(x) #x
```

```
#define STRINGIZE(x) STRINGIZE_(x)

#ifdef __APPLE__
#include <mach-o/getsect.h>

#define DECLARE_LD_(LNAME) extern const unsigned char _section$__DATA__##LN
#define LD_NAME_(LNAME) _section$__DATA__##LNAME
#define LD_SIZE_(LNAME) (getsectbyLNAME("__DATA", "__" STRINGIZE(LNAME))->s
#define DECLARE_LD(LNAME) DECLARE_LD_(LNAME)
#define LD_NAME(LNAME) LD_NAME_(LNAME)
#define LD_SIZE(LNAME) LD_SIZE_(LNAME)

#elif (defined __MINGW32__) /* mingw */

#define DECLARE_LD(LNAME)                                       \
        extern const unsigned char binary_##LNAME##_start[];    \
        extern const unsigned char binary_##LNAME##_end[];
#define LD_NAME(LNAME) binary_##LNAME##_start
#define LD_SIZE(LNAME) ((binary_##LNAME##_end) - (binary_##LNAME##_start))
#define DECLARE_LD(LNAME) DECLARE_LD_(LNAME)
#define LD_NAME(LNAME) LD_NAME_(LNAME)
#define LD_SIZE(LNAME) LD_SIZE_(LNAME)

#else /* gnu/linux ld */

#define DECLARE_LD_(LNAME)                                        \
        extern const unsigned char _binary_##LNAME##_start[];     \
        extern const unsigned char _binary_##LNAME##_end[];
#define LD_NAME_(LNAME) _binary_##LNAME##_start
#define LD_SIZE_(LNAME) ((_binary_##LNAME##_end) - (_binary_##LNAME##_start
#define DECLARE_LD(LNAME) DECLARE_LD_(LNAME)
#define LD_NAME(LNAME) LD_NAME_(LNAME)
#define LD_SIZE(LNAME) LD_SIZE_(LNAME)
#endif

DECLARE_LD(ld_data_bin);

int main() {
        const char* p_data = reinterpret_cast<const char*>(LD_NAME(ld_data_
        // impossible, not null-terminated
        //puts(p_data);
```

```
        // must copy instead
        return 0;
}
```

This scales a little bit better in terms of raw compilation time but is shockingly OS, vendor and platform specific in ways that novice developers would not be able to handle fully. The macros are required to erase differences, lest subtle differences in name will destroy one's ability to use these macros effectively. We omitted the code for handling VC++ resource files because it is excessively verbose than what is present here.

N.B.: Because these declarations are `extern`, the values in the array cannot be accessed at compilation/translation-time.

9.1.4. `incbin`

There is a tool called `incbin` which is a 3rd party attempt at pulling files in at "assembly time". Its approach is incredibly similar to `ld`, with the caveat that files must be shipped with their binary. It unfortunately falls prey to the same problems of cross-platform woes when dealing with Visual C, requiring additional pre-processing to work out in full.

9.1.5. `xxd`, but done Raw

Some people cannot even use the `xxd` tool on their platforms because it cannot be used. This is the case where tools need to be able to package things, and therefore their build tools need to accommodate for not having their information. The way to help save for this is to create other small utilities that effectively duplicate the tools, but in different ways.

This has affected packaging of Debian-style packages on multiple distributions.

9.2. Type Flexibility

Note: As per the vote in the September C++ Evolution Working Group Meeting, Type Flexibility is not being pursued in the preprocessor for various implementation and support splitting concerns.

A type can be specified after the `#embed` to view the data in a very specific manner. This allows data to initialized as exactly that type.

Type flexibility was not pursued for various implementation concerns. Chief among them was single-purpose preprocessors that did not have access to frontend information. This meant it was very hard to make a system that was both preprocessor conformant but did not require e.g. `sizeof(...)` information at the point of preprocessor invocation. Therefore, the type flexibility feature was pulled from `#embed` and will be conglomerated in other additions such as `std::bitcast` or `std::embed`.

```
/* specify a type-name to change array type */
const int shorten_flac[] = {
                #embed int "stripped_music.flac"
};
```

The contents of the resource are mapped in an implementation-defined manner to the data, such that it will use `sizeof(type-name) * CHAR_BIT` bits for each element. If the file does not have enough bits to fill out a multiple of `sizeof(type-name) * CHAR_BIT` bits, then a diagnostic is required. Furthermore, we require that the type passed to `#embed` that must one of the following fundamental types, signed or unsigned, spelled exactly in this manner:

— `char`, `unsigned char`, `signed char`

— `short`, `unsigned short`, `signed short`

— `int`, `unsigned int`, `signed int`

— `long`, `unsigned long`, `signed long`

— `long long`, `unsigned long long`, `signed long long`

More types can be supported by the implementation if the implementation so chooses (both the GCC and Clang prototypes described below support more than this). The reason exactly these types are required is because these are the only types for which there is a suitable way to obtain their size at pre-processor time. Quoting from §5.2.4.2.1, paragraph 1:

> The values given below shall be replaced by constant expressions suitable for use in `#if` preprocessing directives.

This means that the types above have a specific size that can be properly initialized by a preprocessor entirely independent of a proper C frontend, without needing to know more than how to be a preprocessor. Originally, the proposal required that every use of `#embed` is accompanied by a `#include <limits.h>` (or, in the case of C++, `#include <climits>`). Instead, the proposal now lets the implementation "figure it out" on an implementation-by-implementation basis.

§ References

§ Informative References

[CIRCLE-EMBED-TWEET]

Sean Baxter. *@embed added to Circle*. URL: https://twitter.com/seanbax/status/1205195567003045888

[CLANG-LARGE-INIT-BUG]

LLVM Foundation. *Memory Consumption Reduction for Large Array Initialization?*. URL: https://bugs.llvm.org/show_bug.cgi?id=44399

[GCC-LARGE-INIT-BUG-C]

GCC. *[8/9/10 regression] Uses lots of memory when compiling large initialized arrays*. URL: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=12245

[GCC-LARGE-INIT-BUG-CPP]

GCC. *[8/9/10 regression] Uses lots of memory when compiling large initialized arrays*. URL: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=14179

[INCBIN]

Dale Weiler (graphitemaster). *incbin: load files at 'assembly' time*. URL: https://github.com/graphitemaster/incbin

[LLVM-STRING-INIT-FAIL]

Luke Drummond. *[llvm-dev] [tablegen] table readability / performance*. January 14th, 2020. URL: http://lists.llvm.org/pipermail/llvm-dev/2020-January/138225.html

[NONIUS-VISUAL-C-ERROR]

R. Martinho Fernandes. *nonius generated HTML Reporter*. September 1st, 2016. URL: https://github.com/libnonius/nonius/blob/devel/include/nonius/reporters/html_reporter.h%2B%2B#L42

[P1040R6]

JeanHeyd Meneide. *std::embed and #depend*. 29 February 2020. URL: https://wg21.link/p1040r6